

Python Intermediate Concepts – Complete Notes with Examples

a) Class Properties

A **class** is a blueprint for creating objects. It defines the attributes (properties) and methods (functions) that the objects will have.

Creating a Class

```
class Student:  
    name = "Rashik"  
    age = 18
```

Creating an object:

```
s1 = Student()  
  
print(s1.name)  
print(s1.age)
```

Output

```
Rashik  
18
```

Instance Properties

These belong to each object separately.

```
class Student:  
  
    def __init__(self, name, age):  
        self.name = name  
        self.age = age
```

```
s1 = Student("Rashik",18)
s2 = Student("John",20)
```

```
print(s1.name)
print(s2.name)
```

Output

Rashik
John

Class Properties (Class Variables)

Shared by every object.

```
class Student:
```

```
    school = "ABC School"
```

```
    def __init__(self,name):
        self.name = name
```

```
s1 = Student("Ali")
s2 = Student("John")
```

```
print(s1.school)
print(s2.school)
```

Instance Methods

```
class Student:
```

```
    def __init__(self,name):
        self.name = name
```

```
    def display(self):
        print("Student:",self.name)
```

```
s = Student("Rashik")
s.display()
```

b) OOP Concepts

There are four main concepts in Object-Oriented Programming.

1. Encapsulation

Wrapping data and methods together.

```
class Bank:
```

```
    def __init__(self,balance):
        self.balance = balance
```

```
    def deposit(self,amount):
        self.balance += amount
```

```
acc = Bank(1000)
acc.deposit(500)
```

```
print(acc.balance)
```

2. Inheritance

One class acquires properties of another class.

```
class Animal:
```

```
    def sound(self):
        print("Animal makes sound")
```

```
class Dog(Animal):
```

```
    def bark(self):
```

```
        print("Dog barks")

d = Dog()

d.sound()
d.bark()
```

3. Polymorphism

Same method behaves differently.

```
class Cat:

    def sound(self):
        print("Meow")

class Dog:

    def sound(self):
        print("Bark")

for animal in [Cat(),Dog()]:
    animal.sound()
```

4. Abstraction

Hide unnecessary details.

```
from abc import ABC, abstractmethod

class Vehicle(ABC):

    @abstractmethod
    def start(self):
        pass

class Car(Vehicle):

    def start(self):
```

```
        print("Car Started")

c = Car()
c.start()
```

c) Constructor and Destructor

Constructor

Runs automatically when object is created.

```
class Student:

    def __init__(self,name):
        self.name = name
        print("Constructor Called")

s = Student("Rashik")
```

Destructor

Runs automatically before object is destroyed.

```
class Student:

    def __init__(self):
        print("Object Created")

    def __del__(self):
        print("Object Deleted")

s = Student()

del s
```

d) Error Handling

Errors can crash programs.

Use **try-except** to handle them.

Basic Example

```
try:
    print(10/0)

except ZeroDivisionError:
    print("Cannot divide by zero")
```

Multiple Exceptions

```
try:
    num = int(input())
    print(10/num)

except ValueError:
    print("Enter numbers only")

except ZeroDivisionError:
    print("Cannot divide by zero")
```

else

Runs when no exception occurs.

```
try:
    print(10/2)

except:
    print("Error")

else:
    print("Success")
```

finally

Always executes.

```
try:
    print(5/0)

except:
    print("Error")

finally:
    print("Finished")
```

Raising Exceptions

```
age = -1

if age < 0:
    raise ValueError("Age cannot be negative")
```

e) Modules and Packages

Module

A Python file.

maths.py

```
def add(a,b):
    return a+b
```

Another file:

```
import maths
```

```
print(maths.add(5,4))
```

Import Specific Function

```
from maths import add
```

```
print(add(2,3))
```

Alias

```
import numpy as np
```

Package

A folder containing multiple modules.

```
mypackage/
```

```
__init__.py  
maths.py  
string.py
```

Import:

```
from mypackage.maths import add
```

f) Data Structure Built-in Functions

List Functions

```
numbers = [5,3,8]
```


Function	Description	Example
<code>append(x)</code>	Add element at end	<code>numbers.append(10)</code>
<code>extend(list)</code>	Add multiple elements	<code>numbers.extend([1,2])</code>
<code>insert(i,x)</code>	Insert at index	<code>numbers.insert(1,100)</code>
<code>remove(x)</code>	Remove first occurrence	<code>numbers.remove(3)</code>
<code>pop()</code>	Remove last item (or index if given)	<code>numbers.pop()</code>
<code>clear()</code>	Remove all elements	<code>numbers.clear()</code>
<code>index(x)</code>	Find index	<code>numbers.index(8)</code>
<code>count(x)</code>	Count occurrences	<code>numbers.count(5)</code>
<code>sort()</code>	Sort ascending	<code>numbers.sort()</code>
<code>reverse()</code>	Reverse order	<code>numbers.reverse()</code>
<code>copy()</code>	Create shallow copy	<code>new = numbers.copy()</code>

Useful built-in functions:

```
len(numbers)
max(numbers)
min(numbers)
sum(numbers)
sorted(numbers)
any(numbers)
all(numbers)
```

Explanation:

- **len()** → number of elements.
- **max()** → largest element.
- **min()** → smallest element.
- **sum()** → total of numeric elements.
- **sorted()** → returns a new sorted list without changing the original.
- **any()** → True if at least one element is truthy.
- **all()** → True only if every element is truthy.

Example:

```
numbers = [5, 3, 8]

print(len(numbers))
print(max(numbers))
print(min(numbers))
print(sum(numbers))
print(sorted(numbers))
```

Dictionary Functions

```
student = {
    "name": "Rashik",
    "age": 18
}
```

Function	Description
keys()	Returns all keys
values()	Returns all values
items()	Returns key-value pairs
get(key)	Safe lookup
update(dict)	Update dictionary
pop(key)	Remove key
popitem()	Remove last pair
clear()	Remove everything
copy()	Copy dictionary
setdefault(key, default)	Return value if key exists, otherwise insert default value

Example

```
print(student.keys())
print(student.values())
print(student.items())
```

```
print(student.get("name"))

student.update({"city":"Malappuram"})

student.pop("age")
```

Set Functions

```
A = {1,2,3}
B = {3,4,5}
```

Function	Description
<code>add()</code>	Add element
<code>update()</code>	Add multiple elements
<code>remove()</code>	Remove element (error if absent)
<code>discard()</code>	Remove safely (no error if absent)
<code>pop()</code>	Remove random element
<code>union()</code>	Combine sets
<code>intersection()</code>	Common elements
<code>difference()</code>	Elements in first set only
<code>symmetric_difference()</code>	Elements in either set but not both
<code>issubset()</code>	Check subset
<code>issuperset()</code>	Check superset
<code>clear()</code>	Remove all elements

Example

```
print(A.union(B))
print(A.intersection(B))
print(A.difference(B))
```

String Functions

text = "hello world"

Function	Description
upper()	Uppercase
lower()	Lowercase
capitalize()	First letter uppercase
title()	Capitalize every word
swapcase())	Swap uppercase/lowercase
strip()	Remove spaces
lstrip()	Remove left spaces
rstrip()	Remove right spaces
replace()	Replace text
split()	Convert to list
join()	Join list into string
find()	Find index
index()	Find index (raises error if not found)
count()	Count substring
startswith()	Check prefix
endswith()	Check suffix
isalpha()	Letters only
isdigit()	Digits only
isalnum()	Letters and numbers only

<code>islower()</code>	All lowercase
<code>isupper()</code>	All uppercase

Example

```
print(text.upper())  
print(text.replace("world", "Python"))  
print(text.split())
```

Tuple Functions

```
numbers = (5,8,5,10)
```

Function	Description
<code>count()</code>	Count occurrences
<code>index()</code>	Find index

Built-in functions:

```
len(numbers)  
max(numbers)  
min(numbers)  
sum(numbers)  
sorted(numbers)
```

Example

```
print(numbers.count(5))  
print(numbers.index(10))
```

g) List Comprehension

A shorter way to create lists.

Normal

```
square = []
```

```
for i in range(5):  
    square.append(i*i)
```

List comprehension

```
square = [i*i for i in range(5)]
```

With condition

```
even = [i for i in range(20) if i%2==0]
```

Dictionary Comprehension

```
square = {x:x*x for x in range(5)}
```

```
print(square)
```

Condition

```
even_square = {x:x*x for x in range(10) if x%2==0}
```

h) Lambda Functions

Anonymous one-line function.

Normal

```
def square(x):  
    return x*x
```

Lambda

```
square = lambda x:x*x
```

```
print(square(5))
```

Multiple parameters

```
add = lambda a,b:a+b
```

```
print(add(5,3))
```

Sorting

```
students = [("Ali",80),("John",95),("Sara",70)]
```

```
students.sort(key=lambda x:x[1])
```

```
print(students)
```

i) Generator, Iterator, Decorator

Iterator

Objects that can be iterated one value at a time.

```
numbers = [1,2,3]
```

```
it = iter(numbers)
```

```
print(next(it))
```

```
print(next(it))
```

```
print(next(it))
```

Generator

Generates values lazily using **yield**.

```
def count():  
  
    for i in range(5):  
        yield i  
  
g = count()  
  
for i in g:  
    print(i)
```

Advantages

- Uses less memory.
 - Generates values on demand.
 - Ideal for large datasets or infinite sequences.
-

Decorator

Adds functionality without modifying original function.

```
def decorator(func):  
  
    def wrapper():  
        print("Before")  
        func()  
        print("After")  
  
    return wrapper  
  
@decorator  
def greet():  
    print("Hello")  
  
greet()
```

Output

Before
Hello
After

j) Pass, Break, Continue

pass

Placeholder.

```
for i in range(5):
```

```
    if i==3:  
        pass
```

```
    print(i)
```

break

Stops loop immediately.

```
for i in range(10):
```

```
    if i==5:  
        break
```

```
    print(i)
```

continue

Skips current iteration.

```
for i in range(6):
```

```
    if i==3:  
        continue
```

```
    print(i)
```

k) Operators

Arithmetic Operators

+
-
*
/
%
**
//

Example

a = 10

b = 3

print(a+b)
print(a-b)
print(a*b)
print(a/b)
print(a%b)
print(a**b)
print(a//b)

Comparison Operators

==
!=
<
>
<=
>=

Example

print(5==5)

```
print(10>3)
```

Assignment Operators

```
=  
+=  
-=  
*=  
/=   
%=  
**=  
//=
```

Logical Operators

```
and  
or  
not
```

Bitwise Operators

```
&  
|  
^  
~  
<<  
>>
```

Example

```
print(5 & 3)  
print(5 | 3)
```

Membership Operators

in
not in

Example

```
numbers = [1,2,3]
```

```
print(2 in numbers)
```

```
print(5 not in numbers)
```

Identity Operators

is
is not

Example

```
a = [1,2]
```

```
b = a
```

```
c = [1,2]
```

```
print(a is b)
```

```
print(a is c)
```

```
print(a == c)
```

Difference:

- **is** checks whether two variables refer to the **same object** in memory.
 - **==** checks whether two objects have the **same value**.
-

Unary Operator

Works with **one operand**.

```
a = 5
```

```
print(-a)
```

```
print(+a)
```

```
print(not True)
```

Binary Operator

Works with **two operands**.

```
print(5 + 3)
print(10 > 2)
print(4 * 6)
```

Ternary Operator

Short form of an **if-else** statement.

```
age = 18

result = "Adult" if age >= 18 else "Minor"

print(result)
```

Summary

After completing these topics, you should be able to:

- Build classes and create objects.
- Apply the four OOP principles (Encapsulation, Inheritance, Polymorphism, and Abstraction).
- Use constructors and destructors effectively.
- Handle errors gracefully using **try**, **except**, **else**, **finally**, and **raise**.
- Create and import modules and packages.
- Work confidently with Python's built-in data structures and their most useful methods.
- Write concise code using list and dictionary comprehensions.
- Use lambda functions for simple anonymous operations.
- Understand iterators, generators, and decorators.
- Control loops using **pass**, **break**, and **continue**.

- Use Python operators correctly, including arithmetic, comparison, logical, bitwise, identity, membership, unary, binary, and ternary operators.